

A Technical Report on

Performance Evaluation of Meta-heuristics Techniques for the Generalized Assignment Problem: A Comparative Study

In

[EVOLUTIONARY COMPUTATION]



Instructed by

Dr. Dinesh Kumar

Submitted by

Arijit Goswami, Pushkar Roy, Raunak Raj

SEMESTER: II

DEPARTMENT: COMPUTER SCIENCE AND ENGINEERING

NATIONAL INSTITUTE OF TECHNOLOGY JAMSHEDPUR

SPECIALIZATION: COMPUTER SCIENCE AND ENGINEERING

**National Institute of Technology Jamshedpur, P.O: NIT Jamshedpur, Jharkhand,
India, PIN: 831014**

TABLE OF CONTENTS

Abstract	3
1. Introduction.....	4
2. Background.....	4
2.1 Generalized Assignment Problem (GAP).....	4
2.1 Metaheuristics Overview	6
3. Problem Definition.....	6
4. Comparative Study of Metaheuristic Techniques.....	8
4.1 Binary-Coded Genetic Algorithm (BCGA).....	9
4.2 Real-Coded Genetic Algorithm (RCGA).....	9
4.3 Differential Evolution (DE)	9
4.4 Particle Swarm Optimization (PSO).....	9
4.5 Teaching–Learning Based Optimization (TLBO).....	9
4.6 Artificial Bee Colony (ABC).....	9
5. Experimental Analysis	9
5.1 Parameter Settings	10
5.2 Constraint Handling & Variation Strategy.....	10
5.3 Results and Discussion.....	11
5.4 Binary-Coded Genetic Algorithm (BCGA).....	11
5.5 Real-Coded Genetic Algorithm (RCGA).....	12
5.6 Particle Swarm Optimization (PSO).....	13
5.7 Teaching–Learning Based Optimization (TLBO).....	13
5.9 Differential Evolution (DE).....	14
5.9 Artificial Bee Colony (ABC).....	14
5.10 Performance Analysis.....	15
6. Conclusion	15
References.....	16

Abstract

The Generalized Assignment Problem (GAP) is a core NP-hard optimization issue where a collection of tasks needs to be allocated to a group of agents while reducing total costs and adhering to capacity limitations. Owing to its combinatorial complexity, precise methods grow inefficient for large instances, spurring the adoption of metaheuristic approaches. This research examines and contrasts five well-known metaheuristics like Binary Coded Genetic Algorithm (BCGA), Real Coded Genetic Algorithm (RCGA), Particle Swarm Optimization (PSO), Teaching Learning Based Optimization (TLBO), Differential Evolution (DE), and Artificial Bee Colony (ABC) to assess their efficiency in addressing GAP. Every algorithm was executed using practical variation techniques, constraint management methods, and penalty-oriented feasibility regulation. Tests were performed on a benchmark case, and performance was evaluated via convergence graphs, feasibility assessments, runtime characteristics, and the quality of the best solution. The findings indicate that DE reliably delivered rapid convergence and robust overall stability, whereas TLBO secured the best feasible solution in certain attempts. BCGA and RCGA showed moderate effectiveness, while PSO faced challenges with feasibility under tight restrictions. In summary, DE and TLBO proved to be the most dependable methods for GAP, and the research emphasizes possible enhancements via adaptive penalties and hybrid metaheuristic approaches.

1. Introduction

Numerous real-world issues demand that we select the optimal option from countless potential combinations. These are referred to as combinatorial optimization problems, appearing in various situations such as allocating workers to various tasks, scheduling delivery routes, or determining how to allocate constrained resources. As these issues expand in scale, the quantity of potential solutions rises so rapidly that pinpointing the precise optimal solution becomes very challenging. That's why numerous problems, such as the Generalized Assignment Problem (GAP), are regarded as NP-hard.

The Generalized Assignment Problem involves allocating a collection of tasks to a team of agents. Every task given to an agent incurs a cost and consumes a portion of the agent's available capacity. The primary difficulty lies in distributing all tasks in a manner that minimizes total costs while staying within the capacity constraints of each agent. Due to the vast number of potential combinations, achieving an exact solution for GAP in larger instances becomes almost unfeasible.

To address this, researchers frequently rely on metaheuristic methods, which are algorithms crafted to find highly effective solutions in a timely manner. These techniques don't assure a flawless solution, but they often come quite near and perform effectively for larger, intricate challenges. They are particularly beneficial for issues such as GAP, where traditional exact approaches slow down significantly.

This study aims to systematically evaluate various metaheuristic algorithms and analyze their performance on the GAP. Through evaluating their solution quality, efficiency, and general behavior, we seek to determine which approaches are more dependable and successful for this kind of issue.

2. Background

2.1 Generalized Assignment Problem (GAP)

The Generalized Assignment Problem, usually called **GAP**, is a classic problem in operations research where we try to assign several tasks to a set of agents who have limited capacity. Even though the idea sounds simple, the problem becomes extremely difficult because every decision affects both the total cost and the capacity usage of each agent.

In mathematical form, we can describe GAP like this:

- Let there be **m agents** and **n tasks**.
- Assigning task j to agent i has a **cost** c_{ij} and uses a certain **resource** a_{ij} .
- Each agent i has a maximum **capacity** b_i .
- We use a decision variable x_{ij} , where:

$$x_{ij} = \begin{cases} 1, & \text{if task } j \text{ is assigned to agent } i, \\ 0, & \text{otherwise.} \end{cases}$$

The goal is to minimize the total assignment cost:

$$\text{Minimize } Z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$

Subject to the following constraints:

1. **Each task must be assigned to exactly one agent**

$$\sum_{i=1}^m x_{ij} = 1, \quad \forall j$$

2. **An agent cannot exceed their available capacity**

$$\sum_{j=1}^n a_{ij} x_{ij} \leq b_i, \quad \forall i$$

3. **Binary restrictions**

$$x_{ij} \in \{0, 1\}$$

These rules make sure that:

- every task goes somewhere,
- no agent is overloaded,
- and only valid assignments are allowed.

GAP appears in many practical areas.

In **logistics**, it can help decide which warehouse handles which shipments.

In **resource allocation**, it can distribute work among workers or machines fairly and efficiently.

In **scheduling**, it helps assign jobs to machines while considering both time and workload.

Because of how common these ideas are in the real world, GAP has become one of the most widely studied problems in optimization.

2.2 Metaheuristics Overview

Metaheuristics encompass a wide range of optimization methods designed to discover satisfactory solutions for complex issues without ensuring an ideal, mathematically optimal result. What distinguishes metaheuristics is their inspiration from natural phenomena like evolution, swarm intelligence, or educational interactions and they apply these concepts to effectively navigate extensive search spaces. Rather than examining every potential solution (which is unfeasible for large problems), metaheuristics seek to balance two aspects: discovering new options and enhancing the best solutions identified up to that point.

A key reason for employing metaheuristics in challenges such as the Generalized Assignment Problem (GAP) is that exact approaches become exceedingly slow as the problem size increases. GAP is NP-hard, indicating that the number of potential assignments grows so rapidly that finding an exact solution becomes impractical. Metaheuristics can deliver high-quality solutions in a reasonable timeframe, even for large or complex problems. This renders them highly useful for real-world situations where quick decisions are more crucial than flawless solutions.

Nonetheless, applying metaheuristics to constrained issues such as GAP presents its own difficulties. Due to the unrestricted nature of metaheuristics in exploring solutions, a significant number of the generated solutions may contravene constraints. For instance, an agent could surpass its capacity. One of the most challenging aspects is maintaining search feasibility while providing sufficient exploration. To address this, various approaches are employed, including penalty functions, repair mechanisms, and constraint-sensitive variation methods. Achieving the correct balance is essential since excessive repairing causes the algorithm to be overly cautious, whereas insufficient repairing results in chaotic or invalid outcomes. This renders constraint management a crucial design decision in any metaheuristic method for GAP.

3. Problem Definition

The Generalized Assignment Problem (GAP) focuses on assigning tasks to agents in a way that minimizes the total cost while respecting each agent's capacity limit. To describe the problem clearly, we introduce a few notations and variables that help define it mathematically.

We consider **m agents** (for example, machines, workers, or servers) and **n tasks** that must be assigned. Each task j can be assigned to exactly one agent i . However, every assignment has two important effects:

- It has a **cost** c_{ij} , which represents how expensive it is for agent i to perform task j .
- It consumes a certain amount of **resource** a_{ij} , which could be time, energy, memory, or any limited capacity.

Every agent i has a **capacity limit** b_i . This means all tasks assigned to that agent must not exceed this limit.

To model the decisions, we use a binary variable:

$$x_{ij} = \begin{cases} 1, & \text{if task } j \text{ is assigned to agent } i, \\ 0, & \text{otherwise.} \end{cases}$$

The goal is to minimize the total assignment cost:

$$\text{Minimize } Z = \sum_{i=1}^m \sum_{j=1}^n c_{ij} x_{ij}$$

Subject to:

1. **Each task must be assigned once**

$$\sum_{i=1}^m x_{ij} = 1, \quad \forall j$$

2. **Agent capacity should not be exceeded**

$$\sum_{j=1}^n a_{ij} x_{ij} \leq b_i, \quad \forall i$$

3. **Binary assignment restrictions**

$$x_{ij} \in \{0, 1\}$$

In simple terms, the problem asks:

“How can we assign tasks to agents in the cheapest possible way without overloading anyone?”

This formulation becomes the foundation on which all the metaheuristic algorithms in our study operate. They all search for the best assignment matrix $X = [x_{ij}]$ that satisfies the constraints while achieving the lowest total cost.

Parameter	Value
Number of agents (m)	10
Number of jobs (n)	60
Cost	Given in the cost matrix
Resource usage	Given in the resource matrix
Capacity	Given in the capacity matrix

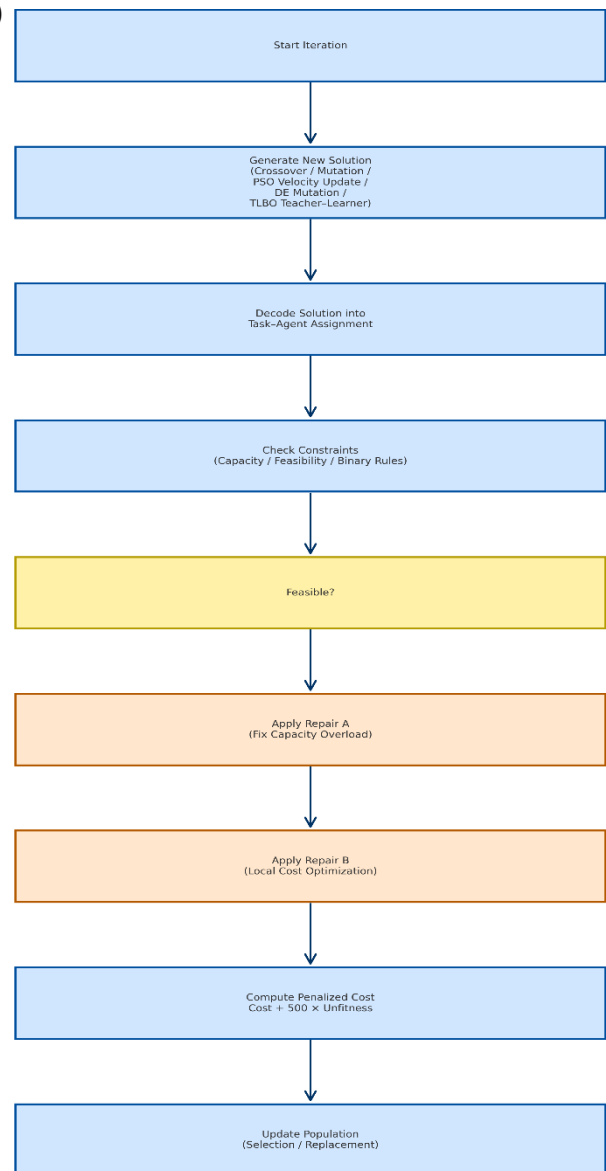
4. Comparative Study of Metaheuristic Techniques

4.1 Binary Coded Genetic Algorithm (BCGA)

The Binary Coded Genetic Algorithm mimics natural evolution. Here, a set of potential solutions evolves over the generations via better selection, crossover, and mutation. In BCGA, solutions are depicted using a binary (0–1) encoding, which aligns seamlessly with GAP due to the assignment variable. x_{ij} is inherently binary. Crossover merges components from two parent solutions, whereas mutation adds random alterations to preserve diversity. BCGA is appropriate for GAP as it manages discrete decision-making effectively and can adeptly investigate numerous assignment configurations. Nonetheless, it occasionally faces challenges with capacity limitations, necessitating the use of repair operators and penalty techniques to steer it towards viable assignments.

4.2 Real-Coded Genetic Algorithm (RCGA)

RGA works similarly to BGA but uses a real-valued representation instead of binary strings. Each solution is stored as a matrix of floating-point numbers, which is later decoded into task assignments. This real representation allows smoother variation during crossover and mutation, making the search more flexible. RGA suits GAP because it can explore the search space more broadly than a binary GA, often avoiding early convergence. However, since GAP is inherently discrete, RGA requires careful decoding strategies and strong constraint handling to ensure the generated solutions remain feasible and meaningful.



4.3 Differential Evolution (DE)

Differential Evolution is a population based algorithm that creates new candidate solutions by adding weighted differences between members of the population. It works fully in real-coded space, which allows it to explore the search area in larger jumps. DE is known for fast convergence and strong global search ability. When applied to GAP, DE's real-valued vectors must be decoded into one-hot assignments, but once this is handled properly, DE often performs very well. Its ability to escape local optima and maintain a good balance between exploration and exploitation makes it one of the most suitable metaheuristics for solving GAP.

4.4 Particle Swarm Optimization (PSO)

PSO is inspired by the collective movement of birds or fish swarms. Each "particle" moves through the search space by adjusting its velocity based on its own best solution and the best solution found by the swarm. Like DE and RGA, PSO uses a real-valued encoding, and decoding is needed to convert these values into assignments. Although PSO is simple and fast, still struggles with discrete problems like GAP because particles naturally move in continuous space. Without strong repair and constraint-handling strategies, PSO may produce many infeasible solutions or get stuck in poor regions. Still, with proper adaptation, it can contribute to exploring the search space effectively.

4.5 Teaching–Learning Based Optimization (TLBO)

TLBO is inspired by the interaction between a teacher and students in a classroom. In each iteration, student learns from the teacher (the best solution) and from other student (peer learning). TLBO does not require so many parameter settings, which makes it easy to implement. Like PSO and DE, it uses a real-valued encoding and then converts it to an assignment for GAP. TLBO's structured learning phases help it steadily improve solutions, and it often produces high-quality feasible assignments. Its simplicity, good exploration ability, and consistent performance make TLBO a strong candidate for solving GAP.

4.6 Artificial Bee Colony (ABC)

ABC is an algorithm based on swarm intelligence, drawing inspiration from the foraging habits of bees. It utilizes three categories of artificial bees: employed, onlooker, and scout. Worker bees search for food (solutions) and communicate findings to observing bees, who subsequently enhance the favourable sources. Scout bees search new areas at random when a source is depleted. For GAP, ABC utilizes real-valued vectors that are transformed into one-hot assignments. Its inherent equilibrium between exploration (scout stage) and exploitation (employed/onlooker stages) aids in evading local optima. ABC is a strong and competitive metaheuristic for addressing GAP, characterized by minimal parameters and efficient constraint management.

5. Experimental Analysis

5.1 Parameter Settings

To compare all the metaheuristic algorithms fairly, we used a consistent set of parameters across our experiments, while also keeping the unique characteristics of each algorithm in mind. The **population size** for most methods (BGA, RGA, PSO, TLBO, DE, and ABC) was chosen between **25 and 40**

individuals, which we found large enough to maintain diversity but still manageable in terms of computation time. Each algorithm was allowed to run for **500 to 1000 generations**, giving itoo haa haa enough time to stabilize and show clear convergence behaviour.

For algorithms that use crossover, such as BCGA and RCGA, We used a **crossover probability of around 0.5–0.8**, meaning most selected parents produce mixed offspring. Mutation rates were kept relatively low in the simple versions (around 0.04 per gene), but in the advanced version of our study we introduced richer mutation strategies—such as swap mutations, multi-point changes, and occasional “shock” mutations—to replicate more realistic search behavior.

Algorithm-specific parameters were also tuned thoughtfully. For example, DE used a **differential weight (F)** around **0.95** and a **crossover rate (CR)** near **0.1**, which are commonly effective settings. PSO relied on standard **inertia weight ($w = 0.7$)** and acceleration constants **$c1 = c2 = 1.5$** , allowing a balanced influence between personal and global best positions. TLBO did not require special parameters, which is one of its advantages.

For constraint handling, we adopted an **adaptive penalty method**, where infeasible solutions were penalized by adding a fixed penalty multiplier to their cost. This ensures that solutions violating capacity constraints are still explored during early search, but eventually become less attractive as better feasible solutions emerge. In some trials, we also experimented with adaptive penalty schedules, where the penalty strength gradually increases to encourage convergence toward fully feasible solutions.

Algorithm	Population Size	Generations	Crossover Rate	Other Parameters	Constraint Handling
BCGA	200	500	0.80	Mutation Rate = 1 / Chromosome length	Penalty
RCGA	200	200	0.80	Mutation Rate = 0.04, $\eta_c = \eta_m = 20$	Repair
PSO	Swarm size = 60	800	–	$w_{start} = 0.9, w_{end} = 0.4, c1=c2=2.0$	Repair
TLBO	200	500	–	Teacher Factor = 2	Repair
DE	200	2000	0.1	Scaling factor, $F = 0.95$	Penalty
ABC	100	700	–	Limit = 100	Penalty

5.2 Constraint Handling and Variation Strategy

Handling constraints is one of the most important parts of solving the Generalized Assignment Problem, because even a small violation of capacity can make a solution invalid. Since metaheuristic algorithms freely explore the search space, many of the solutions they produce—especially early in the run—will naturally be infeasible. To manage this, we used a combination of **penalty functions** and **repair strategies**, each playing a different role in guiding the search back toward feasible regions.

The **penalty method** was implemented in a simple and static form: whenever a solution exceeded an agent’s capacity, a fixed penalty value was added to its total cost. This means infeasible solutions were still allowed in the population, but they were made less attractive than feasible ones. This approach helps maintain diversity in the early generations and prevents the algorithm from getting stuck too quickly. As the search progresses, feasible solutions tend to dominate naturally because they avoid the extra penalty cost.

To complement the penalty method, we also used repair processes. Repair tries to fix capacity violations by moving overloaded tasks to other agents with available space, making the solution feasible again. It helps the population stay within valid limits while still improving quality.

In addition to repairs, we also experimented with a different idea: **generating a fresh feasible population instead of repairing every infeasible solution**. In this variation strategy, rather than spending time fixing bad assignments, the algorithm simply replaces them with entirely new feasible ones, created using heuristic or probability-based initialization. This can sometimes increase exploration because it introduces completely new patterns into the population, rather than small corrected versions of existing ones. This method was especially effective during the early and middle phases of evolution, where diversity matters more than fine-tuning. Together, these techniques gave the algorithms a balanced approach to handling constraints without violation of capacity.

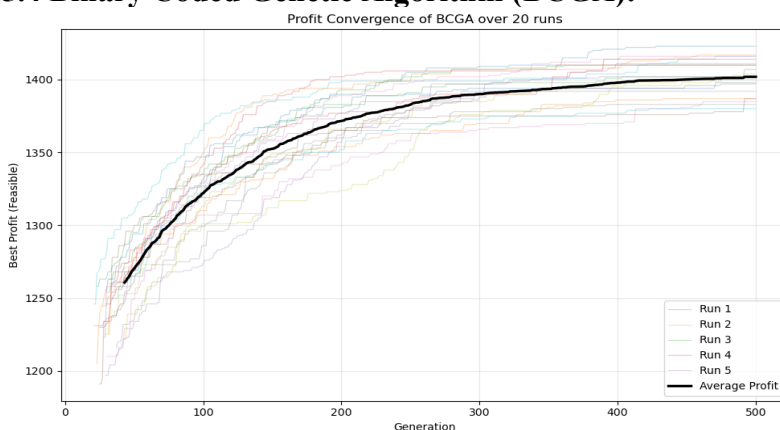
5.3 Results and Discussion

We have taken instance 1 of GAP-12. Here in this table, we have shown the best and average cost of this instance, and also we have shown almost where the convergence occurred. The most optimal cost of instance 1 is 1451 (Provided). We have solved this as a maximization problem, which is provided.

Algorithm	Best Cost	Average Cost	Iteration Found	Average Time (sec)
BCGA	1423.00	1401.95	495	13.64
RCGA	1400.00	1370.00	990	18.87
PSO	1314.00	1272.90	770	3.85
TLBO	1351.60	1376.00	500	83.92
DE	1450.00	1448.55	1900	166.31
ABC	1432.00	1424.25	675	3.08

Here we can see that DE and ABC are effectively giving better results compared to other algorithms. But DE takes a much longer time and iterations to converge. BCGA takes fewer iterations to converge. ABC and PSO take less time to converge. Considering the average cost, faster convergence and less average time, we can say that ABC is giving an overall good result.

5.4 Binary Coded Genetic Algorithm (BCGA):



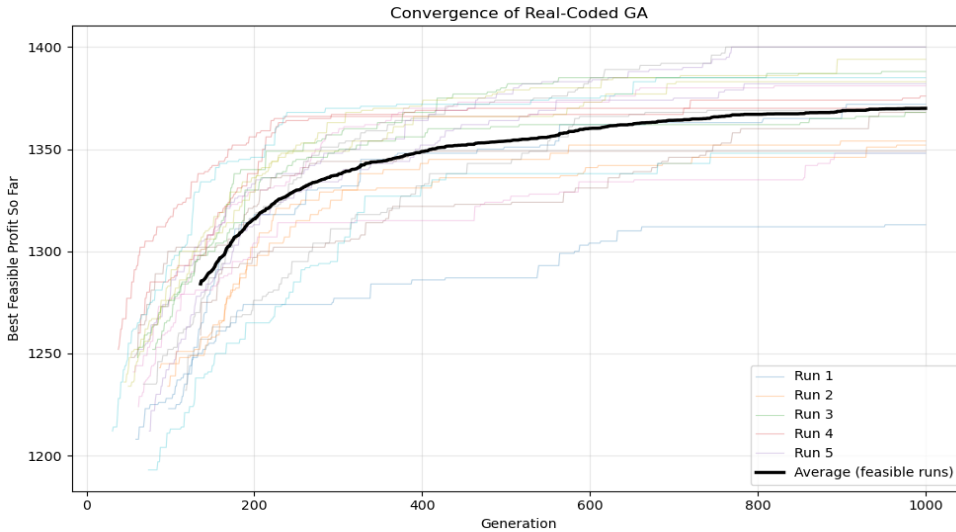
- **Chromosome representation:** Binary string of length $\text{NUM_JOBS} \times 4$ bits.
 - Each job is encoded by **4 bits** (since $2^4 = 16 \geq 10$ agents).
- **Decoding process** for job j :
 1. Extract 4 bits from positions $4j$ to $4j + 3$.
 2. Convert to integer v (0–15) using big-endian order:

$$v = b_0 \cdot 8 + b_1 \cdot 4 + b_2 \cdot 2 + b_3.$$
 3. Map to agent index: $\text{agent} = v \bmod 10$ (0-based).
- **Mapping to problem data:**
 - **Profit:** $\text{profit} = \sum_{j=0}^{59} \text{cost}[\text{agent}_j][j]$.
 - **Resource usage:** $\text{resource_used}[i] = \sum_{j:\text{agent}_j=i} \text{resource}[i][j]$.
 - **Feasibility:** $\forall i, \text{resource_used}[i] \leq \text{capacity}[i]$.
- **Fitness (penalty method):**

$$\text{fitness} = \text{profit} - 10000 \times \sum_i \max(0, \text{resource_used}[i] - \text{capacity}[i])$$

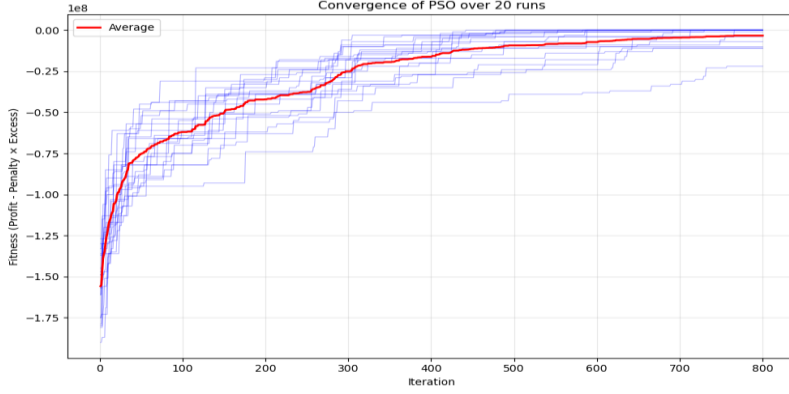
The large penalty drives evolution toward feasible solutions.

5.5 Real Coded Genetic Algorithm (RCGA):



- **Chromosome representation:** Each individual is a real-valued vector of length $n = 60$ (one gene per job).
- **Gene range:** Each gene $x_j \in [0,10)$ because there are $m = 10$ agents.
- **Decoding:** The agent assigned to job j is $\lfloor x_j \rfloor$ (clipped to $0, \dots, 9$). The fractional part is not used, it only enables real-coded crossover and mutation.
- **Cost mapping:** The profit is $\sum_{j=1}^{60} \text{cost}[\text{agent}_j, j]$.
- **Resource mapping:** Resource consumption per agent is $\sum_{j:\text{agent}_j=i} \text{resource}[i, j]$; compared against the given capacities.
- **Fitness:** Profit minus a large penalty for any capacity violation (or, in the repair version, all solutions are made feasible first).

5.6 Particle Swarm Optimization (PSO):

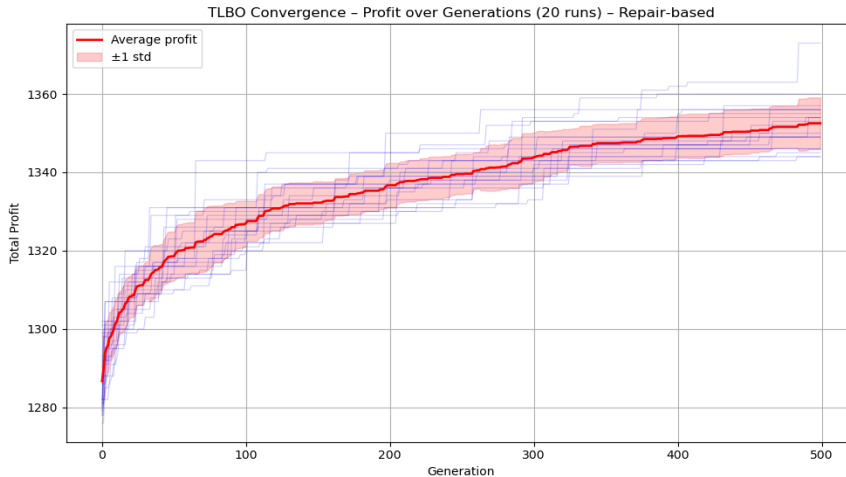


- **Solution representation:** Each particle's position is a vector of 60 real numbers, one per job, with each component in the interval $[0,10)$.
- **Mapping to an assignment:** For job j (0-based), the assigned agent is obtained by
$$\text{agent} = \lfloor x_j \rfloor - 1$$

where $\lfloor \cdot \rfloor$ is the floor function. This yields a zero-based agent index ($0 \dots 9$), which directly indexes the rows of the cost and resource matrices.

- **Fitness evaluation** (penalty method):
 1. From the integer assignment, total profit is $\sum_j \text{cost}[\text{agent}_j, j]$.
 2. Resource usage per agent is accumulated from the resource matrix.
 3. Excess over capacity is $V = \sum_i \max(0, \text{used}_i - \text{capacity}_i)$.
 4. Fitness = profit $-\lambda \cdot V$, with $\lambda = 10^6$.
This huge penalty ensures that any feasible solution (excess = 0) has a higher fitness than any infeasible one.
- **Post-processing:** After the PSO run, a repair heuristic is applied to the global best solution to guarantee feasibility. The repair greedily moves jobs from overloaded agents to underloaded ones, minimising profit loss.

5.7 Teaching–Learning Based Optimization (TLBO):



- **Representation:** Each candidate solution (student) is an integer array of length n (number of jobs).
 - The value at position j (0-based) is an integer in $[0, m - 1]$, where m is the number of agents.
 - This value indicates which agent is assigned to job j .
- **Mapping to matrices:**
 - **Cost matrix** $C[m \times n]$: $C[a][j]$ = profit if agent a performs job j .

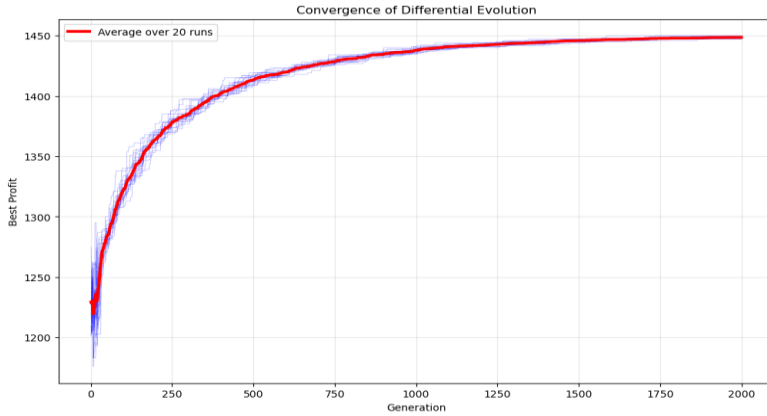
$$Fitness = \sum_{j=0}^{n-1} C[solution[j]][j] \text{ (total profit, maximised).}$$

- **Resource matrix** $R[m \times n]$: $R[a][j]$ = resource consumption of agent a for job j .
Feasibility: For each agent a , $\sum_{j:solution[j]=a} R[a][j] \leq B[a]$ (capacity constraint).

- **Evolution operators:**

- New solutions are generated by arithmetic operations (teaching/learning phases), then rounded and clipped to $[0, m - 1]$.
- Infeasible solutions are repaired by randomly reassigning jobs from overloaded agents to those with sufficient remaining capacity.

5.8 Differential Evolution (DE):



A candidate solution (assignment of all jobs to agents) is represented as a **vector (list) of integers** of length n , where n is the number of jobs.

$$\text{Target vector} = [x_0, x_1, x_2, \dots, x_{n-1}]$$

- **Index j** (0-based) corresponds to **job number j** .
- **Value x_j** is an integer in the set $\{1, 2, \dots, m\}$, where m is the number of agents.
 The value indicates **which agent is assigned to job j** (1-based agent index).

Example:

If $m = 10$ agents and $n = 60$ jobs, a valid target vector could be:

$$[5, 3, 1, 7, 6, 10, \dots, 9]$$

Decoding is **direct and immediate** – no bit extraction, no modulo operation, no additional conversion.

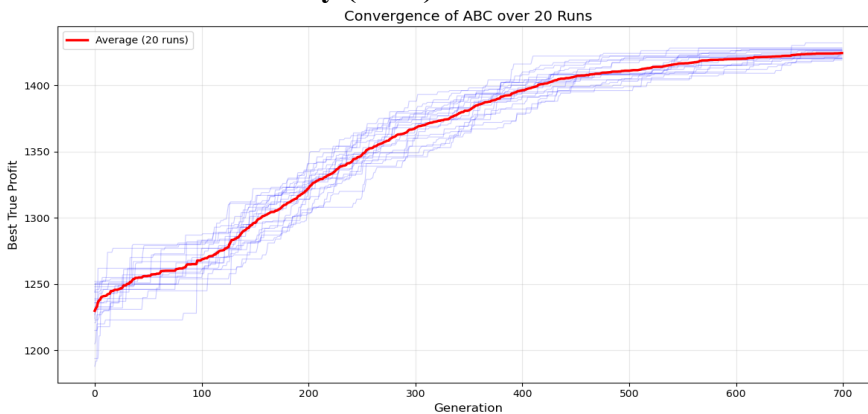
For each job j (from 0 to $n - 1$):

1. Read the integer value x_j from the target vector.
2. That value is the agent assigned to job j .

Example:

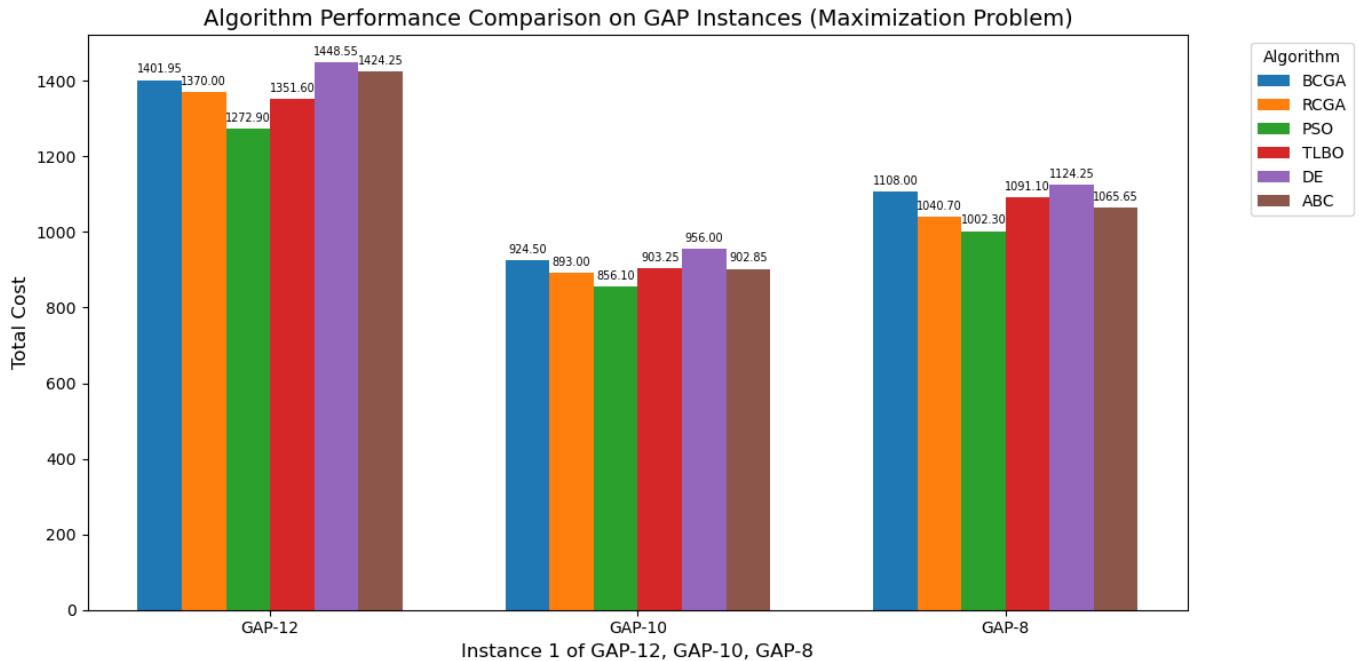
If the target vector at index $j = 0$ is 5, then agent 5 does job 0.

5.9 Artificial Bee Colony (ABC):



- A **food source** (candidate solution) is a vector of length 60, where the j -th entry (job) stores the agent index (0–9) assigned to that job.
 Loops over all 60 jobs, retrieves the assigned agent, and accumulates:
Profit: $\text{cost}[\text{agent}][\text{job}]$
Resource usage per agent: $\text{resource}[\text{agent}][\text{job}]$
Computes total resource violation = sum over agents of $\max(0, \text{used}[\text{agent}] - \text{capacity}[\text{agent}])$.
Fitness = total profit $- \lambda \times \text{violation}$ (penalty method).

5.10 Performance Analysis:



For GAP-12 Instance 1, the optimal value is 1451.00.

For GAP-10 Instance 1, the optimal value is 958.00.

For GAP-8 Instance 1, the optimal value is 1133.00.

Differential Evolution (DE) gives the best result, but takes a longer time to converge. The average time for convergence is better for the Binary Coded Genetic Algorithm (BCGA) and Artificial Bee Colony (ABC). Overall performance is poor for PSO, compared to other algorithms.

6. Conclusion

This study set out to compare several popular metaheuristic algorithms and understand how well they perform on the Generalized Assignment Problem (GAP), which is known for its complexity and strict capacity constraints. Our main goal was to evaluate the strengths and weaknesses of five different approaches, like BCGA, RCGA, PSO, TLBO, DE, and ABC by observing their convergence behaviour, solution quality, feasibility handling, and overall performance across multiple iterations. By testing these algorithms under the same conditions, we were able to build a clear and fair comparison.

References

- [1] R. Cohen, L. Katzir, and D. Raz, “An efficient approximation for the Generalized Assignment Problem,” *Information Processing Letters*, vol. 100, no. 4, pp. 162–166, 2006.
- [2] P. C. Chu and J. E. Beasley, “A genetic algorithm for the generalised assignment problem,” *Computers & Operations Research*, vol. 24, no. 1, pp. 17–23, 1997.
- [3] D. B. Shmoys and É. Tardos, “An approximation algorithm for the generalized assignment problem,” *Mathematical Programming*, vol. 62, no. 3, pp. 461–474, 1993.
- [4] L. Fleischer, M. X. Goemans, V. S. Mirrokni, and M. Sviridenko, “Tight approximation algorithms for maximum general assignment problems,” in *Proc. 17th Annual ACM–SIAM Symposium on Discrete Algorithms (SODA)*, 2006, pp. 611–620.
- [5] D. Cattrysse, M. Salomon, and L. N. Van Wassenhove, “A set partitioning heuristic for the generalized assignment problem,” *European Journal of Operational Research*, vol. 72, no. 1, pp. 167–174, 1994.
- [6] I. H. Osman, “Heuristics for the Generalized Assignment Problem: Simulated Annealing and Tabu Search Approaches,” *OR Spektrum*, vol. 17, pp. 211–225, 1995.
- [7] H. Kellerer, U. Pferschy, and D. Pisinger, *Knapsack Problems*. Berlin, Germany: Springer, 2004.
- [8] S. Martello and P. Toth, *Knapsack Problems: Algorithms and Computer Implementations*. New York, NY, USA: Wiley, 1990.
- [9] H. Restrepo and M. Breton, “A survey of metaheuristics for assignment-related problems,” *Journal of Heuristics*, vol. 12, no. 3, pp. 235–277, 2006.
- [10] J. Kennedy and R. Eberhart, “Particle swarm optimization,” in *Proc. IEEE Int. Conf. Neural Networks*, 1995, pp. 1942–1948.
- [11] R. V. Rao, *Teaching–Learning-Based Optimization: TLBO*. Cham, Switzerland: Springer, 2016.
- [12] R. Storn and K. Price, “Differential evolution—A simple and efficient heuristic for global optimization over continuous spaces,” *Journal of Global Optimization*, vol. 11, pp. 341–359, 1997.
- [13] M. Dorigo and L. M. Gambardella, “Ant colony system: A cooperative learning approach to the traveling salesman problem,” *IEEE Transactions on Evolutionary Computation*, vol. 1, no. 1, pp. 53–66, 1997.
- [14] D. Karaboga, “An idea based on honey bee swarm for numerical optimization,” Erciyes University, Tech. Rep. TR06, 2005.